Selección de arquitecturas y tecnologías de programación

Introducción: Arquitectura Cliente-Servidor **2**
1. Mecanismos de ejecución de código en un navegador web **2**
Flujo típico de ejecución: 2
2. Capacidades y limitaciones de ejecución **3**
Capacidades: 3
Limitaciones: 3
3. Lenguajes de programación en entorno cliente **4**
Otros lenguajes (compilados a JavaScript): 4
4. Tecnologías y lenguajes asociados **4**
APIs del navegador: 5
5. Integración del código con las etiquetas HTML **5**
a) Código embebido en línea 5
b) Bloques <script> en el documento HTML 5
c) Carga de archivos externos 6
d) Vinculación mediante atributos 6
6. Herramientas de programación y prueba sobre clientes web. Librerías y frameworks **6**
Herramientas de desarrollo integradas: 6
Editores de código: 6
Librerías y frameworks: 7

Introducción: Arquitectura Cliente-Servidor

La **arquitectura cliente-servidor** es un modelo fundamental en el desarrollo web. En este modelo, existen dos componentes principales:

- **Cliente**: normalmente es un navegador web que solicita recursos y muestra la información al usuario. Se encarga de la interacción directa con la persona usuaria y ejecuta el código que se le entrega (principalmente HTML, CSS y JavaScript).

- **Servidor**: es el equipo que almacena y procesa los datos. Genera respuestas a las peticiones del cliente, puede ejecutar código en otros lenguajes (Python, PHP, Java, etc.) y accede a bases de datos si es necesario.

El cliente inicia la comunicación enviando una petición (por ejemplo, al abrir una página). El servidor responde con el contenido necesario (HTML, CSS, JavaScript, imágenes...). A partir de ahí, el cliente puede continuar solicitando datos dinámicamente, usando tecnologías como AJAX o Fetch, sin recargar toda la página.

Este modelo es la base del funcionamiento de la web moderna. Dentro de este esquema, es clave entender qué código se ejecuta en el **cliente (navegador)** y qué código se ejecuta en el **servidor**.

---

1. Mecanismos de ejecución de código en un navegador web

Cuando un navegador carga una página web, interpreta su contenido siguiendo un flujo bien definido. En este proceso, se pueden ejecutar distintos tipos de código según el contexto:

- **Código HTML**: es interpretado directamente para construir la estructura de la página. No se ejecuta como tal, sino que se interpreta para crear el DOM (Document Object Model).

- **CSS**: define la presentación visual de los elementos del DOM. También es interpretado.

- **JavaScript**: es el lenguaje de programación ejecutado en el cliente. El navegador incluye un motor de JavaScript (como V8 en Chrome o SpiderMonkey en Firefox) que interpreta y ejecuta este código.

Flujo típico de ejecución:

1. El navegador solicita la página al servidor.

2. Descarga y analiza el HTML.

3. Carga e interpreta los estilos CSS.

4. Encuentra y ejecuta el código JavaScript, en el orden en que aparece en el documento.

5. Si hay eventos (como clics o cambios), el navegador espera y ejecuta el código JavaScript asociado cuando esos eventos se disparan.

   Algunos scripts se pueden ejecutar **antes de que cargue todo el contenido** (`<script>` en `<head>`) o **después de cargar el DOM** (`<script defer>` o colocado al final del `<body>`).

---

2. Capacidades y limitaciones de ejecución

Capacidades:

- **Manipulación del DOM**: los scripts pueden modificar la estructura, contenido y estilo de la página en tiempo real.
- **Gestión de eventos**: permiten reaccionar a acciones del usuario (clics, movimientos del ratón, escritura, etc.).
- **Almacenamiento local**: acceso a `localStorage`, `sessionStorage` o `IndexedDB` para guardar datos en el navegador.
- **Peticiones asincrónicas (AJAX)**: los scripts pueden comunicarse con servidores sin recargar la página.
- **Geolocalización, multimedia y APIs del navegador**: acceso a cámara, micrófono, ubicación, etc. con permisos.

Limitaciones:

- **Seguridad (Sandbox)**: el código JavaScript en el navegador está restringido para evitar acceso a archivos locales o funciones del sistema.
- **Política de mismo origen (Same-Origin Policy)**: limita las peticiones a servidores distintos al origen del documento, salvo con CORS.
- **Rendimiento**: el procesamiento depende del dispositivo del usuario.
- **Persistencia limitada**: los datos guardados en el navegador no son eternos ni completamente seguros.

---

3. Lenguajes de programación en entorno cliente

El **lenguaje principal de programación en entorno cliente es JavaScript**, ya que:

- Es interpretado por todos los navegadores.
- Permite manipular el contenido y comportamiento de la página.

- Interactúa con el DOM y con APIs del navegador.

Otros lenguajes (compilados a JavaScript):

- **TypeScript**: superset de JavaScript que añade tipado estático.
- **Dart**: utilizado en el framework Flutter Web.
- **CoffeeScript**, **Elm**, **ReasonML**: ofrecen distintas ventajas sintácticas o funcionales, pero siempre se **transpilan** a JavaScript.

> Los navegadores **no ejecutan directamente** otros lenguajes como Python, PHP o Java. Este tipo de lenguaje se ejecuta en el servidor.

---

4. Tecnologías y lenguajes asociados

Aparte de JavaScript, hay otros lenguajes y tecnologías clave en el desarrollo cliente:

- **HTML (HyperText Markup Language)**: lenguaje de marcado para definir la estructura del contenido.
- **CSS (Cascading Style Sheets)**: lenguaje de estilos para definir el aspecto visual del contenido.
- **SVG**: lenguaje para gráficos vectoriales escalables.
- **WebAssembly**: permite ejecutar código de lenguajes como C/C++ o Rust en el navegador, a muy alto rendimiento.

APIs del navegador:

JavaScript accede a diversas APIs que amplían su funcionalidad:

- API del DOM
- API de eventos
- API de geolocalización
- API de audio y vídeo (Media API)
- API de notificaciones
- API de almacenamiento local

> Estas tecnologías trabajan juntas para crear experiencias web interactivas, modernas y ricas en contenido.

---

5. Integración del código con las etiquetas HTML

El código JavaScript se puede integrar de varias formas en un documento HTML:

a) Código embebido en línea

```
<button onclick="alert('Hola')">Haz clic</button>
```

b) Bloques `<script>` en el documento HTML

```
<script>
  document.querySelector("button").onclick = function() {
    alert("Hola desde JavaScript");
  };
</script>
```

c) Carga de archivos externos

```
<script src="scripts/app.js" defer></script>
```

- Usar `defer` o `async` controla cuándo se ejecuta el script respecto al DOM.

d) Vinculación mediante atributos

Los atributos como `onclick`, `onchange`, `onmouseover`, etc., permiten asociar funciones JavaScript directamente a elementos.

> **Buena práctica**: separar la lógica JavaScript del HTML y escribir el código en archivos externos.

---

6. Herramientas de programación y prueba sobre clientes web. Librerías y frameworks.

Herramientas de desarrollo integradas:

Los navegadores modernos incluyen herramientas muy potentes:

- **Inspector del DOM**: examina y edita la estructura HTML en tiempo real.
- **Consola JavaScript**: para probar código y ver errores.
- **Debugger**: permite detener la ejecución y examinar el estado del programa.
- **Red (Network)**: inspección de las peticiones y respuestas.
- **Almacenamiento**: ver `localStorage`, `cookies`, etc.

Editores de código:

- **Visual Studio Code**: editor popular con muchas extensiones para desarrollo web.
- **Sublime Text**, **Atom**, **WebStorm**: otras alternativas.

Librerías y frameworks:

- **jQuery**: simplifica la manipulación del DOM, eventos y AJAX.
- **React**: biblioteca para construir interfaces de usuario basadas en componentes.
- **Vue.js**: framework progresivo para interfaces reactivas.
- **Angular**: framework completo para aplicaciones de una sola página (SPA).
- **Bootstrap**: framework CSS con componentes listos y compatibles.

> Estas herramientas ayudan a desarrollar, probar y depurar aplicaciones cliente más rápidamente y con mayor calidad.